Méthodes numériques

Compétences

- ❏ À l'aide d'un langage de programmation, simuler l'évolution temporelle d'un signal généré par un oscillateur.
- ❏ À l'aide d'un langage de programmation, résoudre l'équation de la diffusion thermique à une dimension par une méthode des différences finies dérivée de la méthode d'Euler explicite de résolution des équations différentielles ordinaires.
- ❏ À l'aide d'un langage de programmation, simuler la propagation d'un paquet d'ondes dans un milieu dispersif et visualiser le phénomène d'étalement.

Questions de cours des interrogations orales

- ❏ Sur un exemple précis fourmi par le colleur, mettre une équation différentielle d'ordre 2 sous la forme d'un problème d'Euler. Établir le schéma d'Euler explicite.
- ❏ Établir une approximation numérique de la dérivée seconde spatiale. Sur l'exemple de l'équation de diffusion, établir le schéma permettant de résoudre une équation aux dérivées partielles.

Exercices

❏ Exercice 1 ❏ Exercice 2 ❏ Exercice 3 ❏ Exercice 4 ❏ Exercice 5

Résumé du cours

1. Résolution numérique d'équations différentielles

La méthode d'Euler est une procédure permettant de résoudre numériquement et approximativement des équations différentielles à partir d'une condition initiale.

1.1. Discrétisation

La mémoire des ordinateurs étant finie, il est indispensable de discrétiser le problème pour le résoudre numériquement. Discrétiser consiste à associer à une fonction $y(t)$ une suite $y_i = y(i \cdot \Delta t)$ où Δt est appelé pas de temps. Le pas de temps est l'équivalent de la période d'échantillonnage.

1.2. Problème d'Euler

Un problème d'Euler est une équation différentielle d'ordre 1 munie d'une condition initiale : on cherche la fonction y telle que $y'(t) = f(y,t)$ et $y(t=0) = y_0$. La fonction recherchée peut éventuellement être un vecteur et donc avoir plusieurs composantes.

Exemple 1

$$\frac{\mathrm{d}u}{\mathrm{d}t} = -\frac{1}{\tau}u$$

$$\frac{\mathrm{d}\vec{v}}{\mathrm{d}t} = \vec{g} - \frac{k}{m}\vec{v}$$

Les équations différentielles d'ordre supérieur peuvent être mises sous la forme de problème d'Euler en introduisant un vecteur dont les coordonnées sont des dérivées successives.

Application 1

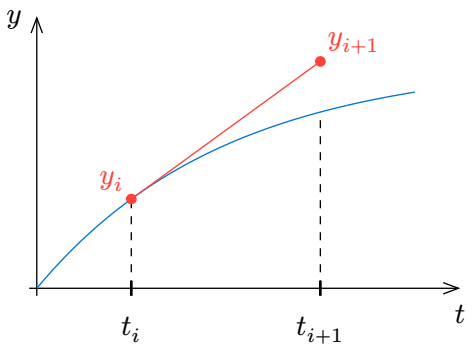
Mettre sous forme de problème d'Euler les équations différentielles suivantes.

$$m\frac{\mathrm{d}^2x}{\mathrm{d}t^2} = -\alpha\frac{\mathrm{d}x}{\mathrm{d}t}^2 - kx \quad \text{avec} \quad \left.\frac{\mathrm{d}x}{\mathrm{d}t}\right|_{t=0} = v_0 \quad \text{et} \quad x(t=0) = x_0$$

$$m\frac{\mathrm{d}^2\overrightarrow{OM}}{\mathrm{d}t^2} = -k\vec{v} + m\vec{g} \quad \text{avec} \quad \left.\frac{\mathrm{d}\overrightarrow{OM}}{\mathrm{d}t}\right|_{t=0} = v_0\vec{e}_x \quad \text{et} \quad \overrightarrow{OM}(t=0) = \vec{0}$$

$$a\frac{\mathrm{d}^3y}{\mathrm{d}t^3} + b\frac{\mathrm{d}^2y}{\mathrm{d}t^2} + c\frac{\mathrm{d}y}{\mathrm{d}t} + dy = e \quad \text{avec} \quad \left.\frac{\mathrm{d}^2y}{\mathrm{d}t^2}\right|_{t=0} = a_0 \ , \quad \left.\frac{\mathrm{d}y}{\mathrm{d}t}\right|_{t=0} = v_0 \quad \text{et} \quad y(t=0) = y_0$$

1.3. Méthode d'Euler



La méthode d'Euler consiste à approximer la courbe localement par sa tangente. Cette approximation s'appuie sur la formule de Taylor à l'ordre 1.

Point clé 1 – Schéma d'Euler ♥ ✍

$$y_{i+1} = y_i + \Delta t \cdot f(y_i, t_i)$$

2. Résolution numérique d'équations aux dérivées partielles

Il est possible d'adapter la méthode d'Euler pour résoudre des équations aux dérivées partielles.

2.1. Discrétisation

Une double discrétisation, spatiale et temporelle, est nécessaire pour résoudre numériquement une équation aux dérivées partielles. On pose $f_{i,j} = f(i \cdot \Delta t, j \cdot \Delta x)$ où Δt est le pas de temps et Δx le pas d'espace.

2.2. Résolution numérique de l'équation de diffusion

Dans cette partie, on s'appuie sur l'exemple de l'équation de diffusion $\frac{\partial T}{\partial t} = D\frac{\partial^2 T}{\partial x^2}$ mais la méthode peut être adaptée à toute équation aux dérivées partielles.

Dans l'équation de diffusion, une dérivée seconde spatiale est présente. On l'approxime en utilisant une formule de Taylor à l'ordre 2.

Point clé 2 – Approximation de la dérivée seconde spatiale ♥ ✍

$$\frac{\partial^2 T}{\partial x^2}_{i,j} \approx \frac{T_{i,j+1} - 2T_{i,j} + T_{i,j-1}}{\Delta x^2}$$

Point clé 3 – Schéma pour résoudre numériquement une équation de diffusion ✍

$$T_{i+1,j} = D\frac{\Delta t}{\Delta x^2}T_{i,j+1} + D\frac{\Delta t}{\Delta x^2}T_{i,j-1} + \left(1 - 2\frac{D(\Delta t)}{\Delta x^2}\right)T_{i,j}$$

Afin que ce schéma soit stable, il est nécessaire que $2D\frac{\Delta t}{\Delta x^2} < 1$.

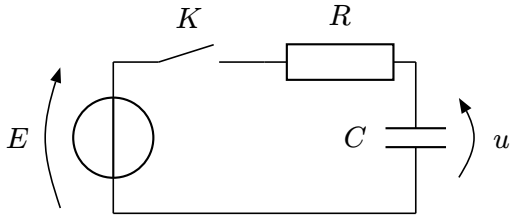
On peut interpréter $T_{i,j}$ sous la forme d'une matrice. Chaque ligne i correspond alors à la température à un instant $i \cdot \Delta t$ partout dans le milieu. Chaque colonne j correspond à la température à une position $j \cdot \Delta x$ à tous les instants.

Exercices

1. 💻 Circuit RC

Notebook Capytale de cet exercice : b4cf-11678437

On cherche à modéliser la charge d'un condensateur dans un circuit RC série soumis à une tension continue E.



À $t = 0$, l'interrupteur K est fermé. Le condensateur est initialement déchargé.

1/ Montrer que u vérifie l'équation différentielle

$$\frac{\mathrm{d}u}{\mathrm{d}t} = -\frac{1}{RC}u + \frac{1}{RC}E$$

2/ Que vaut $u(t = 0^+)$?

On souhaite résoudre numériquement cette équation différentielle à l'aide de la méthode d'Euler explicite. On note $u_i = u(i \cdot \Delta t)$ la tension discrétisée.

3/ Établir une relation de récurrence sur u_i.

4/ Écrire une suite d'instructions permettant de calculer les valeurs successives de u_i. On prendra $R = 1\,\mathrm{k\Omega}$, $C = 1\,\mu\mathrm{F}$, $E = 5\,\mathrm{V}$, $\Delta t = 0{,}2\,\mathrm{ms}$ et on effectuera 25 itérations.

5/ Tracer l'évolution de u en fonction du temps. Comparer avec la solution analytique que vous calculerez et tracerez également.

6/ Observer qualitativement l'effet de la valeur de Δt sur la précision de la solution numérique.

2. 💻 Tir cadré ? ⋆

Notebook Capytale de cet exercice : f85b-11678455

On étudie un tir au football. La vitesse initiale du ballon est de $20\,\mathrm{m\,s^{-1}}$ selon l'axe x (horizontal) et de $12\,\mathrm{m\,s^{-1}}$ selon l'axe z (vertical). Le ballon est sur le sol juste avant le tir.

Dans un premier temps, on ne prend en compte que la gravité.

1/ Établir l'équation différentielle vérifiée par la vitesse $\vec{v}$ du ballon. Exprimer la dérivée de la vitesse.

On résout numériquement l'équation différentielle en utilisant la fonction `solve_ivp` de la bibliothèque `scipy.integrate`.

2/ Compléter le code suivant.

```
from scipy.integrate import solve_ivp, trapezoid
import numpy as np

g = 9.81 # m/s^2
rho = 1.2 # kg/m^3
v0 = np.array([20, 0, 12]) # m/s

def dv_dt(t, v):
    a = ... # accélération
    return a

sol = solve_ivp(dv_dt, [0, 2], v0, max_step=0.01)

```

```
t = sol.t
vx = sol.y[0, :]
vy = sol.y[1, :]
vz = sol.y[2, :]
```

Il est maintenant nécessaire de calculer la position du ballon en intégrant la vitesse en utilisant la méthode des rectangles.

3/ Compléter le code suivant. Attention, les temps calculés par la fonction `solve_ivp` ne sont pas forcément régulièrement espacés.

```
x = [0]
y = [0]
z = [0]

for i in range(1, len(sol.t)):
    x.append(...)
    y.append(...)
    z.append(...)
```

Pour vérifier si le tir est cadré, on trace la trajectoire du ballon grâce au code suivant.

```
fig = plt.figure()
ax = fig.add_subplot(111, projection="3d")

ax.plot(x, y, z)

# Tracé des cages
x_cages = 14.5
y_cages = 0.3
l_cages = 7.32
h_cages = 2.44
ax.plot(
    [x_cages]*4,
    [y_cages, y_cages, y_cages + l_cages, y_cages + l_cages],
    [0, h_cages, h_cages, 0],
    color="orange")

# Mise en forme
ax.set_xlabel("x (m)")
ax.set_ylabel("y (m)")
ax.set_zlabel("z (m)")
ax.set_title("Trajectoire 3D")
ax.set_zlim(0, max(z)*1.1)

plt.tight_layout()
plt.show()
```

4/ Le tir est-il cadré ?

On prend maintenant en compte les frottements avec l'air $\vec{F} = -\frac{1}{2}\pi\rho C_x R^2 \, \|\vec{v}\| \, \vec{v}$. On donne les valeurs numériques suivantes : $\rho = 1{,}2\,\text{kg}\,\text{m}^{-3}$, $C_x = 0{,}47$, $R = 0{,}11\,\text{m}$ et $m = 145\,\text{g}$

5/ Modifier la fonction `dv_dt` pour inclure la force de traînée.

On pourra utiliser la fonction `np.linalg.norm(v)` pour calculer la norme du vecteur vitesse v.

Le tir est-il maintenant cadré ?

Le footballeur a mis de l'effet dans la balle en lui imprimant une rotation de $\Omega = 100\,\text{tr}\,\text{min}^{-1}$ autour de l'axe (Oz). Cette rotation engendre une force de portance appelée force de Magnus et s'exprimant comme $\frac{1}{2}C\rho R^3\vec{\Omega} \wedge \vec{v}$ avec $C \approx 1$.

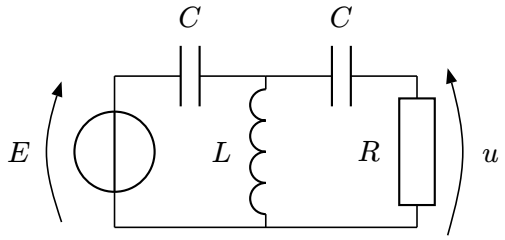
6/ Modifier la fonction `dv_dt` pour inclure la force de Magnus.

Le tir est-il maintenant cadré ?

3. 💻 Circuit électrique d'ordre 3

Notebook Capytale de cet exercice : afd4-11678457

On s'intéresse au circuit électrique d'ordre 3 représenté ci-dessous.



Les valeurs des composants sont $R = 10\,\Omega$, $L = 1 \cdot 10^{-3}\,\mathrm{H}$ et $C = 1 \cdot 10^{-6}\,\mathrm{F}$. La source de tension fournit une tension constante $E = 10\,\mathrm{V}$.

L'évolution de u est régie par l'équation différentielle

$$RLC^2\frac{\mathrm{d}^3u}{\mathrm{d}t^3} + 2LC\frac{\mathrm{d}^2u}{\mathrm{d}t^2} + RC\frac{\mathrm{d}u}{\mathrm{d}t} + u(t) = 0$$

On suppose que les conditions initiales sont $u(t=0) = 0$, $\frac{\mathrm{d}u}{\mathrm{d}t}\big|_{t=0} = 3\,\mathrm{V\,s^{-1}}$ et $\frac{\mathrm{d}^2u}{\mathrm{d}t^2}\big|_{t=0} = 0$. L'objectif de cet exercice est de déterminer l'évolution de $u(t)$ au cours du temps.

1/ On note $v(t) = \frac{\mathrm{d}u}{\mathrm{d}t}$ et $w(t) = \frac{\mathrm{d}^2u}{\mathrm{d}t^2}$. Mettre le problème sous la forme d'un problème d'Euler en exprimant $\frac{\mathrm{d}u}{\mathrm{d}t}$, $\frac{\mathrm{d}v}{\mathrm{d}t}$ et $\frac{\mathrm{d}w}{\mathrm{d}t}$ en fonction de $u(t)$, $v(t)$ et $w(t)$.

2/ On note `Y np.array([u, v, w])`. Définir une fonction `dY_dt(Y,t)` qui retourne le tableau $\frac{\mathrm{d}Y}{\mathrm{d}t}$ en prenant comme entrée le temps t et le tableau Y.

3/ Écrire une suite d'instructions permettant de calculer l'évolution de $Y(t)$ entre $t = 0$ et $t = 1\,\mathrm{ms}$ avec un pas de temps $\Delta t = 1\,\mu\mathrm{s}$ en utilisant la méthode d'Euler explicite.

4/ Tracer l'évolution de $u(t)$ au cours du temps.

La fonction `odeint(func, y0, t)`[1] de la bibliothèque `scipy.integrate` permet de résoudre des problèmes d'Euler en appliquant des méthodes plus sophistiquées mais reposant sur le même principe. Elle prend en entrée

- `func`: une fonction qui retourne la dérivée de l'état en fonction de l'état et du temps,
- `y0`: l'état initial,
- `t`: un tableau des instants où l'on souhaite connaître la solution.

5/ Reprendre la question 3 en utilisant la fonction `odeint` pour calculer l'évolution de $Y(t)$.

4. 💻 Oscillateur de Wien ⋆⋆

Notebook Capytale de cet exercice : 2253-2586522

Pour rappel, un oscillateur de Wien est constitué d'un montage amplificateur non-inverseur et d'un filtre de Wien.

On note

- $u(t)$ la tension en entrée de l'amplificateur non-inverseur (et donc en sortie du filtre de Wien)
- $v(t)$ la tension en sortie de l'amplificateur non-inverseur (et donc en entrée du filtre de Wien)

Les fonctions de transfert du filtre de Wien et de l'amplificateur non-inverseur sont respectivement données par :

$$H_{\mathrm{ANI}}(p) = \frac{V(p)}{U(p)} = \frac{1 + \frac{R_2}{R_1}}{1 + \left(1 + \frac{R_2}{R_1}\right)\frac{\tau}{A_0}p}$$

[1] Documentation complète disponible à l'adresse https://docs.scipy.org/doc/scipy/reference/generated/scipy.integrate.odeint.html

$$H_{\text{Wien}}(p) = \frac{U(p)}{V(p)} = \frac{\frac{1}{3}}{1 + \frac{1}{3}\left(\frac{1}{RCp} + RCp\right)}$$

On pose $w = \frac{\mathrm{d}u}{\mathrm{d}t}$. On cherche à mettre le problème sous la forme d'un problème d'Euler de la forme $\frac{\mathrm{d}\vec{Y}}{\mathrm{d}t} = F\left(t, \vec{Y}\right)$ avec $\vec{Y} = \begin{pmatrix} u \\ v \\ w \end{pmatrix}$.

1/ Exprimer les dérivées de u, v et w uniquement en fonction de u, v, w, R, C, R_1, R_2, A_0 et τ.

Avec Python, on représente le vecteur $\vec{Y}$ par un array numpy à 3 éléments.

2/ Écrire une fonction Python qui prend en entrée Y et renvoie sa dérivée. On pourra supposer les variables R, C, R_1, R_2, A_0 et τ déjà définies.

```
import numpy as np
def F(t, Y):
  """
  t : le temps (inutile ici, mais solve_ivp le fournit)
  Y : array numpy à 3 éléments contenant u, v et w
  Renvoie un array numpy contenant la dérivée de Y par rapport au temps.
  """
  ...
```

Dans un premier temps, on utilise la fonction `scipy.integrate.solve_ivp` qui résout numériquement des équations différentielles mises sous la forme d'un problème d'Euler. Cette fonction utilise des variantes de la méthode d'Euler la rendant plus précise.

La fonction solve_ivp prend en argument

- la fonction `F` définie précédemment
- le couple (t_i, t_f) correspondant à l'intervalle de temps sur lequel on souhaite résoudre l'équation différentielle
- le tableau $\vec{Y}(t_i)$ correspondant aux conditions initiales

La fonction `solve_ivp` renvoie un objet. Si on le stocke dans la variable `solution`,

- `solution.t` contient les temps
- `solution.y` contient les valeurs successives de $\vec{Y}$ à ces temps

3/ Définir et affecter les variables $R = 1\,\text{k}\Omega$, $C = 10\,\text{nF}$, R_1, R_2, $A_0 = 100\,000$ et $\tau = 0{,}01\,\text{s}$ avec des valeurs vraisemblables satisfaisant la condition de démarrage des oscillations.

```
R = ... # Ohm
C = ... # F
R1 = ... # Ohm
R2 = ... # Ohm
A0 = ...
tau = ... # s
```

4/ Définir `Y0` avec de très petites valeurs pour u, v et w (0,0001 par exemple).

```
Y0 = ...
```

5/ Définir tf pour observer une dizaine d'oscillations.

```
tf = ... # s
```

6/ Tracer u et v en fonction du temps.

```
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp

solution = solve_ivp(..., ..., ...)
plt.plot(..., ..., label='u(t)')
plt.plot(..., ..., label='v(t)')
plt.xlabel('Temps (s)')
plt.ylabel('Tension (V)')
plt.legend()
plt.show()
```

7/ Vérifier la condition de démarrage des oscillations.

8/ Vérifier la valeur de la période des oscillations.

Dans la suite, on souhaite se passer de la fonction `solve_ivp` et implémenter nous-même la méthode d'Euler.

On note $Y_i = Y(i \cdot \Delta t)$ où Δt est la durée entre deux échantillons (période d'échantillonnage).

9/ Dans le cas général, exprimer Y_{i+1} en fonction de Y_i, de i, Δt et de la fonction F.

10/ Implémenter la méthode d'Euler pour simuler l'évolution des tensions pour un oscillateur de Wien. Δt sera choisi de sorte qu'il y ait environ 200 échantillons par période.

```
Delta_t = ... # s, environ 200 échantillons par période
N = int(tf / Delta_t) # Nombre d'échantillons
t = np.zeros(N)
Y = np.zeros((N,3))
t[0] = 0
Y[0] = Y0
for i in range(1,N):
    Y[i] = ...
    t[i] = ...
```

11/ Adapter le code précédent pour prendre en compte la saturation de l'ALI.

```
Vsat = 15 # V, tension de saturation de l'ALI
```

5. 💻 Propagation de la chaleur

Notebook Capytale de cet exercice : 1285-5360170

On cherche à modéliser l'évolution de la température dans un barreau cylindrique en aluminium ($D = 99 \cdot 10^{-6}\,\mathrm{m^2\,s^{-1}}$) de longueur $l = 10\,\mathrm{cm}$. L'évolution de la température est donnée par l'équation de diffusion $\frac{\partial T}{\partial t} = D\frac{\partial^2 T}{\partial x^2}$.

On discrétise spatialement la barre en prenant $N_x = 20$ points.

1/ Sachant que le schéma d'Euler est stable ssi $2D\frac{\Delta t}{\Delta x^2} < 1$, quel pas temporel maximal peut-on choisir ? On choisira dans toute la suite $\Delta t = 0{,}1\,\mathrm{s}$. Compléter le code suivant.

```
D = 99e-6
l = 10e-2
N_x = 20
Delta_x = ... # pas spatial
Delta_t = 0.1 # pas temporel
```

2/ La température dans le barreau à chaque instant sera stockée dans une matrice de sorte que $T_{i,j} = T(i \cdot \Delta t, j \cdot \Delta x)$. On souhaite simuler l'évolution de la température durant 4 minutes. Combien de lignes doit comporter la matrice ? Combien de colonnes ? Compléter le code suivant.

```
import numpy as np
T = np.zeros((...,...))
```

3/ On initialise la simulation en supposant la température égale à 298 K dans le barreau au début. Compléter le code suivant.

```
T[...] = 298 # Température initiale de la barre
```

4/ L'extrémité gauche du barreau ($x = 0$) est maintenue à une température de 350 K tandis que son extrémité droite ($x = 10\,\mathrm{cm}$) est maintenue à 298 K. Compléter le code suivant.

```
T[...] = 350 # Température de la barre en x=0
T[...] = 298 # Température de la barre en x=10cm
```

5/ Montrer que l'équation de diffusion peut donner lieu à un schéma d'Euler $T_{i+1,j} = D\frac{\Delta t}{\Delta x^2}T_{i,j+1} + D\frac{\Delta t}{\Delta x^2}T_{i,j-1} + \left(1 - 2D\frac{\Delta t}{\Delta x^2}\right)T_{i,j}$. Compléter le code suivant.

```
for i in range(len(T)-1):
    for j in range(1, N_x-1):
        T[i+1,j] = ...
```

6/ Expliquer le choix des bornes des deux boucles du code précédent.

7/ Tracer sur le même graphe le profil de température dans la barre au bout de 15 s, 30 s, 1 min, 2 min et 4 min. Compléter le code suivant.

```
import matplotlib.pyplot as plt
x = ... # Positions des points de la barre (m)
plt.plot(x, ..., label='t=15s')
plt.plot(x, ..., label='t=30s')
plt.plot(x, ..., label='t=1min')
plt.plot(x, ..., label='t=2min')
plt.plot(x, ..., label='t=4min')
plt.xlabel('Position le long de la barre (m)')
plt.ylabel('Température (K)')
plt.legend()
plt.show()
```

8/ Tracer la température du point central de la barre en fonction du temps. Compléter le code suivant.

```
t = ... # Instants simulés (s)
plt.plot(t, ...)
plt.xlabel('Temps (s)')
plt.ylabel('Température au centre de la barre (K)')
plt.show()
```